Topics on Computer Science

C++ Exercises

B.Sc - Mobile Robotics

daniel.mcguiness@mci.edu

Daniel T. McGuiness, PhD

MCI

It's not a bug; it's an undocumented feature.
- Anonymous

Version: 2.012

Questions

1 Introductions, Operators, Variables and Expressions

1.1 Write a program to print `Hello, World!` on screen.

1.2 Write a program to display the following output using a single `cout` statement.

Subject	Marks
Mathematics	90
Computer	77
Chemistry	69

1.3 Write a program which accept two numbers and print their sum.

1.4 Write a program which accept temperature in Celsius (C) and prints it in Fahrenheit (F).

1.5 Write a program which accepts a character and display its ASCII value. **Tip:** You may need to use `static_cast<int>()` for this program.

> **ASCII** (*American Standard Code for Information Interchange*) - a character encoding standard for electronic communication. ASCII codes represent text in computers, telecommunications equipment, and other devices. Because of technical limitations of computer systems at the time it was invented, ASCII has just 128 code points, of which only 95 are printable characters, which severely limited its scope. Modern computer systems have evolved to use Unicode, which has millions of code points, but the first 128 of these are the same as the ASCII set.

1.6 Write a program to swap the values of two variables.

2 Flow Control

2.1 Any integer is presented as an input by the user. Please write a program to find out whether this integer is an odd number or even number.

2.2 Find the absolute value ($|n|$) of a number entered by the user.

2.3 If the ages of Sam, Alice and Bob are input by the user, write a code to determine the youngest.

2.4 Write a program to check whether a triangle is valid or not, when the three angles of the triangle are entered by the user. In a Euclidean space, a triangle is valid if the sum of all the three angles is equal to 180 degrees.

2.5 Any year is input by the user. Write a program to determine whether the year is a leap year or not. (If the year is divisible by 4 it is a leap year unless that year is also divisible by 100. i.e., 1904 is a leap year but 1900 is not. The exception is that if the year is also divisible by 400 it is a leap year. i.e., 1900 is not a leap year but 2000 is.)

2.6 Write a program to find the roots of and quadratic equation of type $ax^2 + bx + c$ where a is not equal to zero.

> **Quadratic Equation** - of the form $ax^2 + bx + c$, and has the following solution:
>
> $$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a},$$
>
> where a, b, c are the coefficients of the quadratic equation.Be aware the roots can be **imaginary**.

3 Library Functions

3.1 Write a program which input principal, rate and time from user and calculate compound interest. **Tip:** Use library function `<cmath>`.

$$A = P\left(1 + \frac{r}{n}\right)^{nt}.$$

where A is the final amount, P is the initial value, r is the interest rate, n is the number of times interest applied per time period and t is the number of time periods elapsed.

3.2 Write a program to compute area of triangle. Sides of the triangle (a, b, c) are given by the user.

$$\text{Area} = \sqrt{(s \times (s-a) \times (s-b) \times (s-c))}, \quad \text{where} \quad s = (a+b+c)/2.$$

3.3 Write a code to test an entered value is a letter (upper or lower), digit or special (i.e., ! or ?) using library functions. **Tip:** Use library function `<cctype>`.

3.4 Write a program which display a number between 10 to 100 randomly using a pseudorandom generator function. **Tip:** Use library functions `<cstdlib>` and `<ctime>`.

3.5 Write a program to implement the number guessing game. In this game, the computer chooses a random number between 1 and 100, and the player tries to guess the number in as few attempts as possible. Each time the player enters a guess, the computer tells him whether the guess is too high, too low, or right. Once the player guesses the number, the game is over. **Tip:** Use library functions `<cstdlib>` and `<ctime>`.

4 User Defined Functions

4.1 Write a program using function which accept two integers as an argument and return its sum. Call this function from `main()` and print the results in `main()`.

4.2 Write a function that receives two numbers as an argument and display all prime numbers between these two numbers and call this function from `main()`.

4.3 Write a function to calculate the factorial value of any integer as an argument and call this function from `main()` and print the results in `main()`.

Factorial - the factorial of a non-negative integer n, denoted by $n!$, is the product of all positive integers less than or equal to n. The factorial of n also equals the product of n with the next smaller factorial $n-1$:

$$n! = n \times (n-1) \times (n-2) \ldots \times 2 \times 1,$$

$$n! = n \times (n-1)!.$$

4.4 Raising a number n to a power p is the same as multiplying n by itself p times.

$$n^p = \underbrace{n \times n \times \ldots \times n}_{p \text{ times}}$$

Write a function called `power()` that takes two arguments; a `double` value for n and an `int` value for p and return the result as double. Use default argument of 2 for p, so that if this argument is omitted the number will be squared. Write the main which gets value from the user to test power function.

4.5 Write a function called `zeroSmall()` that has two integer arguments being passed by reference and sets the smaller of the two numbers to 0. Write the `main()` to access the function.

5 One Dimensional Arrays

5.1 Write a program to find the sum and average of one dimensional (1-D) integer array.

5.2 Write a program to swap first and last element of an integer 1-D array.

5.3 Write a program to reverse the element of an integer 1-D array.

5.4 Write a program to find the largest and smallest element of an array.

5.5 Write a menu driven program with following options:

- Accept elements of an array,
- Display elements of an array,
- Sort the array using insertion sort method,
- Sort the array using selection sort method,
- Sort the array using bubble sort method.

Write functions for all options. The functions should have two parameters;

- name of the array,
- elements in the array.

6 Strings

6.1 Write a program to find the length of string. (i.e., length of `"Servus"` is 6)

6.2 Write a program to display string from backward (i.e., `"Servus"` $\rightarrow$ `"suvreS"`).

6.3 Write a program to count number of words in string (i.e., there are 4 words in the sentence "Time to learn programming".)

6.4 Write a program to concatenate one string contents to another.

6.5 Write a program to compare two strings they are exact equal or not.

6.6 Write a program to check a string is palindrome or not. (i.e., `"racecar"` is a palindrome as writing it forwards or backwards yields the same result.)

7 Two Dimensional Arrays

7.1 Write user defined functions for square matrix to calculate:

- Left diagonal sum,
- Right diagonal sum.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \text{where} \quad \begin{array}{l} \text{Left diagonal sum: } a_{11} + a_{22} + a_{33} \\ \text{Right diagonal sum: } a_{31} + a_{22} + a_{13} \end{array}$$

7.2 Write a user defined functions named `upper()` and `lower()` which takes a two-dimensional array A, with size n rows and m columns as argument and prints the upper half and lower half of the array. For example;

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \texttt{upper()} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix} \quad \text{and} \quad \texttt{lower()} = \begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

7.3 Write a function which accepts a 2D array of integers and its size as arguments and displays the elements of middle row and the elements of middle column, assuming the 2D Array to be a square matrix with odd dimension (i.e. 3-by-3, 5-by-5, 7-by-7).

7.4 Write a program to multiply array A and B of order $n \times l$ and $l \times m$.

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}}_{A} \times \underbrace{\begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}}_{B} = \underbrace{\begin{bmatrix} a_{11} \times b_{11} + a_{12} \times b_{21} & a_{11} \times b_{12} + a_{12} \times b_{22} \\ a_{21} \times b_{11} + a_{22} \times b_{21} & a_{21} \times b_{12} + a_{22} \times b_{22} \end{bmatrix}}_{A \times B}$$

8 Classes & Objects

8.1 Define a `class Student` with the following specifications:

Private members of class Student:

`administration_number`	integer
`student_name`	20 character float
`english, maths, science`	float
`total`	float
`calculatedTotal()`	calculate english + maths + science with float return type.

Public member function of class student

`takeData()`	Function to accept values for `administration_number`, `student_name`, `english`, `maths`, `science` and invoke `calculatedTotal()` to calculate total.
`showData()`	Function to display all the data members on the screen.

8.2. Define a `class Batsman` with the following specifications:

Private members of class Batsman:

`batsman_code`	6 digit number
`batsman_name`	20 character float
`number_of_timeouts, runs`	integer
`batting_average`	calculated as `runs / number_of_timeouts`
`calculatedAverage()`	Function to compute `batting_average`

Public member function of class Batsman:

`readData()`	Function to accept values for `batsman_code`, `batsman_name`, and `number_of_timeouts` and invoke `calculatedAverage()` to calculate average.
`displayData()`	Function to display all the data members on the screen.

8.2 Define a `class Examination` with the following specifications:

Private members of class Examination:

`test_code`	integer
`description`	string
`number_of_candidates`	integer
`requiredCentres()`	integer
`calculatedCentres()`	Calculated as `number_of_candidates` / 100 + 1

Public member function of class Examination:

`shedule()`	allow user to enter values for `test_code`, `description`, `number_of_candidates` and call function `requiredCentres()` to calculate the number of centres.
`showData()`	Function to display all the data members on the screen.

8.3 Define a `class Book` with the following specifications:

Private members of class Book:

`book_number`	integer
`book_title`	maximum 20 characters
`book_price`	float
`totalCost()`	integer
`calculatedCentres()`	function to calculate the total cost for n number of copies where n is passed to the function as argument

Public member function of class Book:

`input()`	to read `book_number`, `book_title` and `book_price`
`purchase()`	function to ask the user to input the number of copies to be purchased. It invokes `totalCost()` and prints the total cost to be paid by the user.

8.4 Define a `class Report` with the following specifications:

Private members of class Report:

`admission_number`	4 digit number
`student_name`	maximum 20 characters
`marks`	an array of 5 floating values
`average`	average of the marks
`getAverage()`	function to calculate the mark average

Public member function of class Report:

`readInformation()`	to read `admission_number`, `student_name` and `marks` and invokes the `getAverage()` to be purchased. It invokes `totalCost()` and prints the total cost to be paid by the user.
`showData()`	Function to display all the data members on the screen.

8.5 Write a program that defines a `class Shape` with a constructor that gives value to `width` and `height`. The define two sub-classes: `class Triangle` and `class Rectangle`, that calculate the area of the shape using `area()`. In the `main()`, define two variables: a triangle, and a rectangle. Then call the `area()` function for these two variables.

8.6 Write a program with a `class Mother` and an inherited `class Daughter`. Both of them should have a method `void display()` that prints a message (different for mother and daughter). In the `main()` define a daughter and call the `display()` method on it.

9 Text File Handling

9.1 Write a program to write number 1 to 100 in a data file `notes.txt`. **Tip:** You can use library function `<fstream>`.

9.2 Write a program, which initializes a string variable to the content "It was the best of times, it was the worst of times." and outputs the string to the file `string_file.txt`. **Tip:** You can use library function `<fstream>`.

9.3 Write a program to read the content from a text file `text.txt`, count and display the number of characters present in it. **Tip:** You can use library function `<fstream>`.

9.4 Write a program to count the number of blank present in a text file named `blank_spaces.txt`. **Tip:** You can use library functions `<fstream>` and `<string>`.

9.5 Write a function to count number of words in a text file named `words.txt`. **Tip:** You can use library functions `<fstream>` and `<string>`.

9.6 Write a function to print the 10 most frequent words in the text file `story.txt`. **Tip:** Use the following libraries: `<fstream>`, `<string>`, `<map>`, `<vector>`, `<algorithm>` and `<cctype>`.

10 Pointers

10.1 Write a program that asks the user to enter integers as inputs to be stored in the variables `k` and `l` respectively. There are also two integer pointers named `ptrK` and `ptrL`. Assign the values of `k` and `l` to `ptrK` and `ptrL` respectively and display them.

10.2 Write a program to find the maximum value of an integral data set. The program will ask the user to input the number of data values in the set and each value. The program prints on screen a pointer that points to the maximum value.

11 Library Information

`<iostream>` Header that defines the standard input/output stream objects. ( )

`<cmath>` Set of functions to for common mathematical operations. ( )

`<cctype>` Set of functions to classify and transform individual characters. ( )

`<cstdlib>` Defines several general purpose functions, including dynamic memory management, random number generation, communication with the environment, integer arithmetic, searching, sorting and converting. ( )

`<ctime>` Function definitions to get and manipulate date and time information. ( )

`<fstream>` Provides file stream classes ( )

`<string>` Introduces string types, character traits and a set of converting functions ( )

Answers

1 Introductions, Operators, Variables and Expressions

1.1 Write a program to print `Hello, World!` on screen.

```cpp
#include<iostream> // to access io functions

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

1.2 Write a program to display the following output using a single `cout` statement.

Subject	Marks
Mathematics	90
Computer	77
Chemistry	69

```cpp
#include<iostream> // to access io functions

int main() {
    std::cout << "Subject " << "\tMarks" << "\nMathematics\t"
              << 90 << "\nComputer\t" << 77 << "\nChemistry\t" << 69;
    return 0;
}
```

1.3 Write a program which accept two numbers and print their sum.

```cpp
#include<iostream> // to access io functions

int main() {
    int a, b, c;    // create variables to store three values
    std::cout << "\nEnter first number : ";
    std::cin >> a;
    std::cout << "\nEnter second number : ";
    std::cin >> b;
    c = a + b; // calculate the sum
    std::cout << "\nThe Sum is : " << c;
    return 0;
}
```

1.4 Write a program which accept temperature in Celsius (C) and prints it in Fahrenheit (F).

```
#include<iostream> // to access io functions

int main() {
    double fahrenheit;  // store the fahrenheit value
    double celsius = 0.0; // set initial celsius value
    std::cout << "\nEnter temperature in Fahrenheit : ";
    std::cin >> fahrenheit; // gets fahrenheit value
    celsius = (fahrenheit - 32.0) * 5.0 / 9.0;  // calculate celsius value
    std::cout << "Temperature in Celsius is : " << celsius;
    return 0;
}
```

1.5 Write a program which accepts a character and display its ASCII value. **Tip:** You may need to use `static_cast<int>()` for this program.

```
#include<iostream> // to access io functions

int main() {
    char ch;    // store a character value
    std::cout << "\nEnter any character: ";
    std::cin >> ch; // gets the input character value
    std::cout << "ASCII equivalent is: " << static_cast<int>(ch) << std::endl;
    return 0;
}

```

1.6 Write a program to swap the values of two variables.

```
#include <iostream>

int main() {
    int a = 2, b = 3;
    std::cout << "Before swapping a = " << a << " , b = " << b << std::endl;
    // temporary variable
    int temp;
    temp = a; a = b; b = temp;
    std::cout << "After swapping a = " << a << " , b = " << b << std::endl;
    return 0;
}
```

2 Flow Control

2.1 Any integer is presented as an input by the user. Please write a program to find out whether this integer is an odd number or even number.

```cpp
#include<iostream> // to access io functions

int main() {
    int a;  // define a parameter to store value
    std::cout << "Enter any number : ";
    std::cin >> a;
    // if the number is divisible by 2 it is defined as even
    // else the number must be odd.
    if (a % 2 == 0) // state the condition
        std::cout << "The number is even";
    else
        std::cout << "The number is odd";

    return 0;
}
```

2.2 Find the absolute value ($|n|$) of a number entered by the user.

```cpp
#include<iostream> // to access io functions

int main() {
    int a;  // define a variable to store value
    std::cout << "Enter any number:";
    std::cin >> a; // gets value

    // checks whether the number is bigger or smaller than 0
    if (a > 0)
        std::cout << "The absolute value of number is:" << a;
    else
        std::cout << "The absolute value of number is:" << -(a);

    return 0;
}
```

2.3 If the ages of Sam, Alice and Bob are input by the user, write a code to determine the youngest.

```
#include<iostream> // to access io functions

int main() {
    int Sam_age, Alice_age, Bob_age;
    std::cout << "Enter Sam age:";
    std::cin >> Sam_age;
    std::cout << "Enter Alice age:";
    std::cin >> Alice_age;
    std::cout << "Enter Bob age:";
    std::cin >> Bob_age;

    if (Sam_age < Alice_age && Sam_age < Bob_age)
        std::cout << "Sam is the youngest";
    else if (Alice_age < Sam_age && Alice_age < Bob_age)
        std::cout << "Alice is the youngest";
    else
        std::cout << "Bob is the youngest";

    return 0;
}
```

2.4 Write a program to check whether a triangle is valid or not, when the three angles of the triangle are entered by the user. In a Euclidean space, a triangle is valid if the sum of all the three angles is equal to 180 degrees.

```
#include<iostream>

int main() {
    int angle1, angle2, angle3;
    std::cout << "Enter the three angles of triangle:";
    std::cin >> angle1 >> angle2 >> angle3;

    return (angle1 + angle2 + angle3 == 180) ? 1 : 0;
}
```

2.5 Any year is input by the user. Write a program to determine whether the year is a leap year or not. (If the year is divisible by 4 it is a leap year unless that year is also divisible by 100. i.e., 1904 is a leap year but 1900 is not. The exception is that if the year is also divisible by 400 it is a leap year. i.e., 1900 is not a leap year but 2000 is.)

```
#include<iostream>

int main() {
    int year; std::cout << "Enter the year : "; std::cin >> year;

    if ((year % 400 == 0 || year % 100 != 0) && (year % 4 == 0))
        std::cout << "It is a leap year";
    else
        std::cout << "It is not a leap year";
    return 0;
}
```

2.6 Write a program to find the roots of and quadratic equation of type $ax^2 + bx + c$ where a is not equal to zero.

```
#include <iostream>
#include <cmath>

int main() {
    float a, b, c, d, root1, root2;
    std::cout << "Enter value of  a, b and c : "; std::cin >> a >> b >> c;
    d = b * b - 4 * a * c;

    if (d == 0) {
        root1 = (-b) / (2 * a); root2 = root1;
        std::cout << "Roots are real & equal";
    } else if (d > 0) {
        root1 = -(b + sqrt(d)) / (2 * a); root2 = -(b - sqrt(d)) / (2 * a);
        std::cout << "Roots are real & distinct";
    } else {
        root1 = (-b) / (2 * a); root2 = sqrt(-d) / (2 * a);
        std::cout << "Roots are imaginary";
    }
    std::cout << "\nRoot 1= " << root1 << "\nRoot 2= " << root2; return 0;
}
```

3 Library Functions

3.1 Write a program which input principal, rate and time from user and calculate compound interest. **Tip:** Use library function `<cmath>`.

$$A = P\left(1 + \frac{r}{n}\right)^{nt}.$$

where A is the final amount, P is the initial value, r is the interest rate, n is the number of times interest applied per time period and t is the number of time periods elapsed.

```
#include<iostream>
#include<cmath>

int main()
{
    float P,R,T,CI;
    std::cout<<"Enter Principal (P), Rate (R) and Time (T): ";
    std::cin>>P>>R>>T;
    CI=P*pow((1+R/100),T) - P;
    std::cout<<"Compound Interest is: "<<CI;
    return 0;
}
```

3.2 Write a program to compute area of triangle. Sides of the triangle (a, b, c) are given by the user.

$$\text{Area} = \sqrt{(s \times (s-a) \times (s-b) \times (s-c))}, \quad \text{where} \quad s = (a+b+c)/2.$$

```
#include<iostream>
#include<cmath>

int main() {
    float a, b, c, s, Area;
    std::cout << "Enter three sides of the triangle : ";
    std::cin >> a >> b >> c;
    s = (a + b + c) / 2;
    Area = sqrt(s * (s - a) * (s - b) * (s - c));
    std::cout << "Area of the triangle is: " << Area;

    return 0;
}
```

3.3 Write a code to test an entered value is a letter (upper or lower), digit or special (i.e., ! or ?) using library functions. **Tip:** Use library function `<cctype>`.

```
#include<iostream>
#include<cctype>

int main() {
    char ch;
    std::cout << "Enter any character :";
    ch = getchar();
    if (isalpha(ch))
        std::cout << "Alphabet";
    else if (isdigit(ch))
        std::cout << "Number";
    else
        std::cout << "Special Character";

    return 0;
}
```

3.4 Write a program which display a number between 10 to 100 randomly using a pseudorandom generator function. **Tip:** Use library functions `<cstdlib>` and `<ctime>`.

```
#include<iostream>
#include<ctime>

int main() {
    int n;
    srand(time(0));
    n = rand() % 91 + 10;
    std::cout << "The randomly selected number is :" << n;
    return 0;
}
```

3.5 Write a program to implement the number guessing game. In this game, the computer chooses a random number between 1 and 100, and the player tries to guess the number in as few attempts as possible. Each time the player enters a guess, the computer tells him whether the guess is too high, too low, or right. Once the player guesses the number, the game is over. **Tip:** Use library functions `<cstdlib>` and `<ctime>`.

```
#include <iostream>
#include <cstdlib>
#include <ctime>

int main() {
    int num, guess, tries = 0;
    srand(time(0)); //seed random number generator
    num = rand() % 100 + 1; // random number between 1 and 100
    std::cout << "Guess My Number Game\n\n";

    do {
        std::cout << "Enter a guess between 1 and 100 : ";
        std::cin >> guess;
        tries++;

        if (guess > num)
            std::cout << "Too high!\n\n";
        else if (guess < num)
            std::cout << "Too low!\n\n";
        else
            std::cout << "\nCorrect! You got it in " << tries << " guesses!\n";
    } while (guess != num);

    return 0;
}
```

4 User Defined Functions

4.1 Write a program using function which accept two integers as an argument and return its sum. Call this function from `main()` and print the results in `main()`.

```
#include<iostream>

int sum(int a, int b) { int total; total = a + b; return total; }

int main() {
    int x, y, s;
    std::cout << "Enter first number : "; std::cin >> x;
    std::cout << "Enter second number : "; std::cin >> y;
    s = sum(x, y); std::cout << "The sum is : " << s;
    return 0;
}

```

```

```

4.2 Write a function that receives two numbers as an argument and display all prime numbers between these two numbers and call this function from `main()`.

```
#include<iostream>

void showPrime(int, int);

int main() {
    int x, y;
    std::cout << "Enter first  number : "; std::cin >> x;
    std::cout << "Enter second  number : "; std::cin >> y;
    showPrime(x, y);
    return 0;
}

void showPrime(int a, int b) {
    bool flag;
    for (int i = a + 1; i <= b; i++) {
        flag = false;
        for (int j = 2; j < i; j++) { if (i % j == 0)  { flag = true; break; } }
        if (flag == false && i > 1){ std::cout << i << std::endl; }
    }
}
```

4.3 Write a function to calculate the factorial value of any integer as an argument and call this function from `main()` and print the results in `main()`.

```
#include<iostream>

int factorial(int a) {
    int fact = 1;
    while (a >= 1) {
        fact = fact * a;
        a--;
    }
    return fact;
}

int main() {
    int x, f;
    std::cout << "Enter number : ";
    std::cin >> x;
    f = factorial(x);
    std::cout << "The factorial is :" << f;
    return 0;
}


```

4.4 Raising a number n to a power p is the same as multiplying n by itself p times.

$$n^p = \underbrace{n \times n \times \ldots \times n}_{p \text{ times}}$$

Write a function called `power()` that takes two arguments; a `double` value for n and an `int` value for p and return the result as double. Use default argument of 2 for p, so that if this argument is omitted the number will be squared. Write the main which gets value from the user to test power function.

```
#include<iostream>

double power(double, int= 2);

int main() {
    int p;
    double n, r;
    std::cout << "Enter number : ";
    std::cin >> n;
    std::cout << "Enter exponent : ";
    std::cin >> p;
    r = power(n, p);
    std::cout << "Result is " << r;
    r = power(n);
    std::cout << "\nResult without passing exponent is " << r;
    return 0;
}

double power(double a, int b) {
    double x = 1;
    for (int i = 1; i <= b; i++) { x = x * a; }

    return x;
}
```

4.5 Write a function called `zeroSmall()` that has two integer arguments being passed by reference and sets the smaller of the two numbers to 0. Write the `main()` to access the function.

```
#include<iostream>

using namespace std;

void zero_small(int &, int &);

int main() {
    int x, y;
    cout << "Enter first number : ";
    cin >> x;
    cout << "Enter second number : ";
    cin >> y;
    zero_small(x, y);
    cout << "First number is : " << x;
```

```
    cout << "\nSecond number is : " << y;

    return 0;
}

void zero_small(int &a, int &b) {
    if (a < b)
        a = 0;
    else
        b = 0;
}
```

5 One Dimensional Arrays

5.1 Write a program to find the sum and average of one dimensional (1-D) integer array.

```
#include<iostream>

int main() {

    int Arr[100], n, i, sum = 0;
    std::cout << "Enter number of elements you want to insert: "; std::cin >> n;

    for (i = 0; i < n; i++) {
        std::cout << "Enter element " << i + 1 << ": "; std::cin >> Arr[i];
    }
    for (i = 0; i < n; i++) { sum += Arr[i]; }

    std::cout << "\nThe sum of Array is: " << sum;
    std::cout << "\nThe average of Array is: " << sum / i;
    return 0;
}
```

5.2 Write a program to swap first and last element of an integer 1-D array.

```
#include<iostream>

int main() {

    int Arr[100], n, i, temp;
    std::cout << "Enter number of elements you want to insert: "; std::cin >> n;

    for (i = 0; i < n; i++) {
        std::cout << "Enter element " << i + 1 << ":"; std::cin >> Arr[i];
    }

    temp = Arr[0]; Arr[0] = Arr[n - 1]; Arr[n - 1] = temp;
    std::cout << "\nArray after swapping" << std::endl;

```

```
    for (i = 0; i < n; i++){ std::cout << Arr[i] << " "; }

    return 0;
}
```

5.3 Write a program to reverse the element of an integer 1-D array.

```
#include<iostream>

int main() {
    int Arr[100], n, temp, i, j;
    std::cout << "Enter number of elements you want to insert: "; std::cin >> n;

    for (i = 0; i < n; i++) {
        std::cout << "Enter element " << i + 1 << ":"; std::cin >> Arr[i];
    }
    for (i = 0, j = n - 1; i < n / 2; i++, j--) {
        temp = Arr[i]; Arr[i] = Arr[j]; Arr[j] = temp;
    }

    std::cout << "\nReverse array" << std::endl;
    for (i = 0; i < n; i++){ std::cout << Arr[i] << " ";}
    return 0;
}
```

5.4 Write a program to find the largest and smallest element of an array.

```
#include<iostream>

int main() {
    int Arr[100], n, i, small, large;
    std::cout << "Enter number of elements you want to insert: "; std::cin >> n;

    for (i = 0; i < n; i++) {
        std::cout << "Enter element: " << i + 1 << ":"; std::cin >> Arr[i];
    }

    small = Arr[0]; large = Arr[0];

    for (i = 1; i < n; i++) {
        if (Arr[i] < small){ small = Arr[i]; }
        if (Arr[i] > large){ large = Arr[i];}
    }
    std::cout << "\nLargest is: " << large << ", Smallest is: " << small;
    return 0;
}
```

5.5 Write a menu driven program with following options:

- Accept elements of an array,
- Display elements of an array,

- Sort the array using insertion sort method,
- Sort the array using selection sort method,
- Sort the array using bubble sort method.

Write functions for all options. The functions should have two parameters;

- name of the array,
- elements in the array.

```
#include<iostream>

void accept(int Arr[], int s) {
    for (int I = 0; I < s; I++) {
        std::cout << "Enter element " << I + 1 << ":"; std::cin >> Arr[I];
    }
}
void display(int Arr[], int s) {
    std::cout << "The elements of the array are:\n";
    for (int I = 0; I < s; I++){ std::cout << Arr[I] << " "; }
}
void insertionSort(int Arr[], int s) {
    int I, J, Temp;
    for (I = 1; I < s; I++) { Temp = Arr[I]; J = I - 1;
        while ((Temp < Arr[J]) && (J >= 0)) { Arr[J + 1] = Arr[J]; J--; }
        Arr[J + 1] = Temp;
    }
}
void selectionSort(int Arr[], int s) {
    int I, J, Temp, Small;
    for (I = 0; I < s - 1; I++) { Small = I;
        for (J = I + 1; J < s; J++)   //finding the smallest element
            if (Arr[J] < Arr[Small]){ Small = J; }

        if (Small != I) { Temp = Arr[I]; Arr[I] = Arr[Small]; Arr[Small] = Temp; }
    }
}

void bubbleSort(int Arr[], int s) {
    int I, J, Temp;
    for (I = 0; I < s - 1; I++) {
        for (J = 0; J < (s - 1 - I); J++)
            if (Arr[J] > Arr[J + 1]) {
                Temp = Arr[J];    //swapping
                Arr[J] = Arr[J + 1];
                Arr[J + 1] = Temp;
            }
    }
}

int main() {
    int Arr[100], n, choice;
    std::cout << "Enter Size of Array "; std::cin >> n;
    do {
```

```
        std::cout << "\n\nMENU";
        std::cout << "\n1. Accept elements of array";
        std::cout << "\n2. Display elements of array";
        std::cout << "\n3. Sort the array using insertion sort method";
        std::cout << "\n4. Sort the array using selection sort method";
        std::cout << "\n5. Sort the array using bubble sort method";
        std::cout << "\n6. Exit";
        std::cout << "\n\nEnter your choice 1-6 :";
        std::cin >> choice;

        switch (choice) {
            case 1: { accept(Arr, n); break; }
            case 2: { display(Arr, n); break; }
            case 3: { insertionSort(Arr, n); break; }
            case 4: { selectionSort(Arr, n); break;}
            case 5: { bubbleSort(Arr, n); break;}
            case 6: { break;}
            default: { std::cout << "\nInvalid choice"; }
        }
    } while (choice != 6);

    return 0;
}

```

6 Strings

6.1 Write a program to find the length of string. (i.e., length of `"Servus"` is 6)

```
#include<iostream>

int main() {
    char str[80];
    std::cout << "Enter string: "; std::cin.getline(str, 80);
    int i; //Hold length of string
    for (i = 0; str[i] != '\0'; i++); std::cout << "Length of string is: " << i;
    return 0;
}
```

6.2 Write a program to display string from backward (i.e., `"Servus"` → `"suvreS"`).

```
#include<iostream>

int main() {
    char str[80];
    std::cout << "Enter string: "; std::cin.getline(str, 80);
    int l; //Hold length of string
    for (l = 0; str[l] != '\0'; l++); //Find the length of the string
    //Display the string backwards
```

```
    for (int i = l - 1; i >= 0; i--) { std::cout << str[i]; }
    return 0;
}
```

6.3 Write a program to count number of words in string (i.e., there are 4 words in the sentence "Time to learn programming".)

```
#include<iostream>

int main() {
    char str[80];
    std::cout << "Enter a string: "; std::cin.getline(str, 80);
    int words = 0; // Holds number of words
    for (int i = 0; str[i] != '\0'; i++) { if (str[i] == ' ') { words++; } }
    std::cout << "The number of words = " << words + 1 << std::endl;
    return 0;
}
```

6.4 Write a program to concatenate one string contents to another.

```
#include<iostream>

int main() {
    char str1[80], str2[80];
    int l = 0; //Hold length of first string
    std::cout << "Enter first string: "; std::cin.getline(str1, 80);
    std::cout << "Enter second string: "; std::cin.getline(str2, 80);
    //Find length of first string.
    for (l = 0; str1[l] != '\0'; l++);
    //Adding second string content in first
    for (int i = 0; str2[i] != '\0'; i++) { str1[l++] = str2[i]; }

    str1[l] = '\0';
    std::cout << "\nThe first string after adding second string content:\n\n" << str1;
    return 0;
}
```

6.5 Write a program to compare two strings they are exact equal or not.

```
#include<iostream>

using namespace std;

int main() {
    char str1[80], str2[80];

    cout << "Enter first string: "; gets(str1);
    cout << "Enter second string: "; gets(str2);

```

```
    int i;
    for (i = 0; str1[i] == str2[i] && str1[i] != '\0' && str2[i] != '\0'; i++);

    if (str1[i] - str2[i] == 0)
        cout << "Strings are equal";
    else
        cout << "Strings are not equal";

    return 0;
}
```

6.6 Write a program to check a string is palindrome or not. (i.e., `"racecar"` is a palindrome as writing it forwards or backwards yields the same result.)

```
#include<iostream>

using namespace std;

int main() {
    char str[80];

    cout << "Enter string: ";
    cin.getline(str, 80);

    int l; //Hold length of string

    //finding length of string
    for (l = 0; str[l] != '\0'; l++);

    //Comparing first element with last element till middle of string
    int i;
    for (i = 0; (i < l / 2) && (str[i] == str[l - i - 1]); i++);

    if (i == l / 2)
        cout << "Palindrome";
    else
        cout << "Not a palindrome";

    return 0;
}
```

7 Two Dimensional Arrays

7.1 Write user defined functions for square matrix to calculate:

- Left diagonal sum,
- Right diagonal sum.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \text{where} \quad \begin{array}{l} \text{Left diagonal sum: } a_{11} + a_{22} + a_{33} \\ \text{Right diagonal sum: } a_{31} + a_{22} + a_{13} \end{array}$$

```cpp
#include<iostream>

int main() {
    int a[5][5], d1sum = 0, d2sum = 0, m, i, j;
    std::cout << "Enter size of the square matrix(max 5):" << std::endl;
    std::cin >> m;
    std::cout << "\nEnter the Matrix row wise:\n" << std::endl;

    for (i = 0; i < m; i++) {
        for (j = 0; j < m; ++j) {
            std::cin >> a[i][j];
        }
    }

    for (i = 0; i < m; ++i) {
        for (j = 0; j < m; ++j) {
            if (i == j)
                d1sum += a[i][j];
            if (i + j == (m - 1))
                d2sum += a[i][j];
        }
    }

    std::cout << "\nSum of left diagonal is " << d1sum << std::endl;
    std::cout << "\nSum of right diagonal is " << d2sum << std::endl;

    return 0;
}
```

7.2 Write a user defined functions named `upper()` and `lower()` which takes a two-dimensional array A, with size n rows and m columns as argument and prints the upper half and lower half of the array. For example;

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \texttt{upper()} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix} \quad \text{and} \quad \texttt{lower()} = \begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

```cpp
#include<iostream>

void lower(int matrix[10][10], int m) {
    int i, j;
    for (i = 0; i < m; i++) { for (j = 0; j < m; j++) {
            if (i < j) { std::cout << "0" << " "; }
```

```
            else { std::cout << matrix[i][j] << " "; }
        }
        std::cout << std::endl;
    }
}

// Function to form upper triangular matrix
void upper(int matrix[10][10], int m) {
    int i, j;
    for (i = 0; i < m; i++) { for (j = 0; j < m; j++) {
            if (i > j) { std::cout << "0" << " "; }
            else { std::cout << matrix[i][j] << " "; }
        }
        std::cout << std::endl;
    }
}

int main() {
    int m, i, j, matrix[10][10];
    std::cout << "Enter square matrix size (max 10):" << std::endl; std::cin >> m;
    std::cout << "\nEnter the Matrix row wise:\n" << std::endl;
    for (i = 0; i < m; i++) { for (j = 0; j < m; ++j) { std::cin >> matrix[i][j]; } }
    std::cout << "Lower triangular matrix: \n" << std::endl; lower(matrix, m);
    std::cout << "Upper triangular matrix: \n" << std::endl; upper(matrix, m);
    return 0;
}
```

7.3 Write a function which accepts a 2D array of integers and its size as arguments and displays the elements of middle row and the elements of middle column, assuming the 2D Array to be a square matrix with odd dimension (i.e. 3-by-3, 5-by-5, 7-by-7).

```
#include<iostream>

int main() {
    int size;
    std::cout << "Enter the order matrix (The order should be in odd dimensions): ";
    std::cin >> size;
    int A[size][size];
    for (int i = 0; i < size; i++)   //taking input
    {
        for (int j = 0; j < size; j++) {
            std::cout << "Enter the " << i + 1 << j + 1
                      << " value of Matrix : ";
            std::cin >> A[i][j];
        }
    }

    std::cout << "Middle Row is: ";   // printing the mid row
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
```

```
            if (i == size / 2)
                std::cout << A[i][j] << "\t";
        }

    }
    std::cout << "\nMiddle Column is:";    // printing the mid col
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            if (j == size / 2)
                std::cout << A[i][j] << "\t";
        }
    }
}
```

7.4 Write a program to multiply array A and B of order $n \times l$ and $l \times m$.

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}}_{A} \times \underbrace{\begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}}_{B} = \underbrace{\begin{bmatrix} a_{11} \times b_{11} + a_{12} \times b_{21} & a_{11} \times b_{12} + a_{12} \times b_{22} \\ a_{21} \times b_{11} + a_{22} \times b_{21} & a_{21} \times b_{12} + a_{22} \times b_{22} \end{bmatrix}}_{A \times B}$$

```
#include <iostream>

int main() {
    int a[10][10], b[10][10], mult[10][10], r1, c1, r2, c2, i, j, k;

    std::cout << "Rows and columns for first matrix: ";
    std::cin >> r1 >> c1;
    std::cout << "Rows and columns for second matrix: ";
    std::cin >> r2 >> c2;

    // If column of first matrix in not equal to row of second matrix,
    // ask the user to enter the size of matrix again.
    while (c1 != r2) {
        std::cout << "Error! column of first matrix not equal to row of second.";
        std::cout << "Enter rows and columns for first matrix: ";
        std::cin >> r1 >> c1;
        std::cout << "Enter rows and columns for second matrix: ";
        std::cin >> r2 >> c2;
    }

    // Storing elements of first matrix.
    std::cout << std::endl << "Enter elements of matrix 1:" << std::endl;
    for (i = 0; i < r1; ++i)
        for (j = 0; j < c1; ++j) {
            std::cout << "Enter element a" << i + 1 << j + 1 << " : ";
            std::cin >> a[i][j];
        }

    // Storing elements of second matrix.
    std::cout << std::endl << "Enter elements of matrix 2:" << std::endl;
    for (i = 0; i < r2; ++i)
```

```
        for (j = 0; j < c2; ++j) {
            std::cout << "Enter element b" << i + 1 << j + 1 << " : ";
            std::cin >> b[i][j];
        }

    // Initializing elements of matrix mult to 0.
    for (i = 0; i < r1; ++i)
        for (j = 0; j < c2; ++j) { mult[i][j] = 0; }

    // Multiplying matrix a and b and storing in array mult.
    for (i = 0; i < r1; ++i)
        for (j = 0; j < c2; ++j)
            for (k = 0; k < c1; ++k) { mult[i][j] += a[i][k] * b[k][j]; }

    // Displaying the multiplication of two matrix.
    std::cout << std::endl << "Output Matrix: " << std::endl;
    for (i = 0; i < r1; ++i)
        for (j = 0; j < c2; ++j) {
            std::cout << " " << mult[i][j];
            if (j == c2 - 1) { std::cout << std::endl; }
        }
    return 0;
}
```

8 Classes & Objects

8.1 Define a `class Student` with the following specifications:

Private members of class Student:

`administration_number`	integer
`student_name`	20 character float
`english, maths, science`	float
`total`	float
`calculatedTotal()`	calculate english + maths + science with float return type.

Public member function of class student

`takeData()`	Function to accept values for `administration_number`, `student_name`, `english`, `maths`, `science` and invoke `calculatedTotal()` to calculate total.
`showData()`	Function to display all the data members on the screen.

8.2. Define a `class Batsman` with the following specifications:

Private members of class Batsman:

`batsman_code`	6 digit number
`batsman_name`	20 character float
`number_of_timeouts, runs`	integer
`batting_average`	calculated as `runs / number_of_timeouts`
`calculatedAverage()`	Function to compute `batting_average`

Public member function of class Batsman:

`readData()`	Function to accept values for `batsman_code`, `batsman_name`, and `number_of_timeouts` and invoke `calculatedAverage()` to calculate average.
`displayData()`	Function to display all the data members on the screen.

```cpp
#include <iostream>

class student {

private:

    int administration_number;
    char student_name[20];
    float english, maths, science;
    float total;

    float calculatedTotal() { return english + maths + science; }

public:

    void takeData() {

        std::cout << "Enter admission number: ";
        std::cin >> administration_number;
        std::cout << "Enter student student_name: ";
        std::cin >> student_name;
        std::cout << "Enter marks in english, maths, science: ";
        std::cin >> english >> maths >> science;

        total = calculatedTotal();
    }

    void showData() {
        std::cout   << "Admission number: " << administration_number
                    << "\nStudent student_name: " << student_name
                    << "\nEnglish: " << english << "\nMath: " << maths
                    << "\nScience: " << science << "\nTotal: " << total;
    }
};

int main() {
    student obj; obj.takeData(); obj.showData();
```

```
    return 0;
}
```

8.2 Define a `class Examination` with the following specifications:

Private members of class Examination:

`test_code`	integer
`description`	string
`number_of_candidates`	integer
`requiredCentres()`	integer
`calculatedCentres()`	Calculated as `number_of_candidates` / 100 + 1

Public member function of class Examination:

`shedule()`	allow user to enter values for `test_code`, `description`, `number_of_candidates` and call function `requiredCentres()` to calculate the number of centres.
`showData()`	Function to display all the data members on the screen.

```
#include <iostream>

class Batsman {
    int batsman_code;
    char batsman_name[20];
    int runs, number_of_timeouts;
    float batting_average;
    void calculatedAverage() {
        batting_average = (runs / number_of_timeouts);
    }

public:
    void readData();
    void displayData();
};

void Batsman::readData() {
    std::cout << "Enter Batsman code: ";
    std::cin >> batsman_code;
    std::cout << "Enter Batsman student_name: ";
    std::cin >> batsman_name;
    std::cout << "Enter runs scored and number of time outs ";
    std::cin >> runs >> number_of_timeouts;
    calculatedAverage();
}

void Batsman::displayData() {

    std::cout << "Batsman code: " << batsman_code
              << "\nBatsman student_name: " << batsman_name
```

```
                << "\nRuns Scored: " << runs
                << "\nNumber of Timeouts: " << number_of_timeouts
                << "\nBatting Average " << batting_average;
}

int main() {

    Batsman obj;

    obj.readData();
    obj.displayData();

    return 0;
}
```

8.3 Define a `class Book` with the following specifications:

Private members of class Book:

`book_number`	integer
`book_title`	maximum 20 characters
`book_price`	float
`totalCost()`	integer
`calculatedCentres()`	function to calculate the total cost for n number of copies where n is passed to the function as argument

Public member function of class Book:

`input()`	to read `book_number`, `book_title` and `book_price`
`purchase()`	function to ask the user to input the number of copies to be purchased. It invokes `totalCost()` and prints the total cost to be paid by the user.

```
#include <iostream>
#include <string>

class Examination {

private:
    int test_code;
    std::string description;
    int number_of_candidates;
    int requiredCentres;
    int calculatedCentres() { return number_of_candidates / 100 + 1; }

public:
    void schedule();
    void showData();
```

```
};

void Examination::schedule() {

    std::cout << "Enter Test code: ";
    std::cin >> test_code;
    std::cout << "Enter no of candidates: ";
    std::cin >> number_of_candidates;
    std::cin.ignore();
    std::cout << "Enter description: " << std::endl;
    getline(std::cin, description);
    requiredCentres = calculatedCentres();
}

void Examination::showData() {

    std::cout << "Test code: " << test_code
              << "\nDescripton: " << description
              << "\nNo of candidate: " << number_of_candidates
              << "\nCenter required: " << requiredCentres;
}

int main() {
    Examination obj;
    obj.schedule(); obj.showData();
    return 0;
}
```

8.4 Define a `class Report` with the following specifications:

Private members of class Report:

`admission_number`	4 digit number
`student_name`	maximum 20 characters
`marks`	an array of 5 floating values
`average`	average of the marks
`getAverage()`	function to calculate the mark average

Public member function of class Report:

`readInformation()`	to read `admission_number`, `student_name` and `marks` and invokes the `getAverage()` to be purchased. It invokes `totalCost()` and prints the total cost to be paid by the user.
`showData()`	Function to display all the data members on the screen.

```cpp
#include <iostream>
#include <string>

class Book {
    int book_number;
    std::string book_title;
    float book_price;

    void totalCost(int N) {
        float total_cost;
        total_cost = book_price * N;
        std::cout << total_cost;
    }

public:

    void input() {
        std::cout << "Enter Book Number ";
        std::cin >> book_number;
        std::cin.ignore();
        std::cout << "Enter Book Title ";
        getline(std::cin, book_title);
        std::cout << "Enter price per copy ";
        std::cin >> book_price;
    }

    void purchase() {
        int n;
        std::cout << "Enter number of copies to purchase ";
        std::cin >> n;
        std::cout << "Total cost is ";
        totalCost(n);
    }
};

int main() {
    Book obj;
    obj.input();
    obj.purchase();
    return 0;
}
```

8.5 Write a program that defines a `class Shape` with a constructor that gives value to `width` and `height`. The define two sub-classes: `class Triangle` and `class Rectangle`, that calculate the area of the shape using `area()`. In the `main()`, define two variables: a triangle, and a rectangle. Then call the `area()` function for these two variables.

```cpp
#include <iostream>
#include <string>

```

```cpp
class Report {
    int admission_number; std::string student_name;
    float marks[5], average;
    void getAverage() {
        average = (marks[0] + marks[1] + marks[2] + marks[3] + marks[4]) / 5;
    }

public:
    void readInformation(); void showData();
};

void Report::readInformation() {
    do {
        std::cout << "Enter 4-digit admission number: ";
        std::cin >> admission_number;
    } while (admission_number < 999 || admission_number > 10000);
    std::cin.ignore();
    std::cout << "Enter student_name: ";
    getline(std::cin, student_name);
    std::cout << "Enter marks in: ";

    for (int i = 0; i < 5; i++) {
        std::cout << "Subject: " << i + 1 << ":";
        std::cin >> marks[i];

    };
    getAverage();
}

void Report::showData() {
    std::cout << "Admission number: " << admission_number << " Name: " << student_name
              << " Marks are: " << marks[0] << " " << marks[1] << " " << marks[2]
              << " " << marks[3] << " " << marks[4] << " Average: " << average;
}

int main() {
    Report obj; obj.readInformation(); obj.showData();
    return 0;
}
```

8.6 Write a program with a `class Mother` and an inherited `class Daughter`. Both of them should have a method `void display()` that prints a message (different for mother and daughter). In the `main()` define a daughter and call the `display()` method on it.

```cpp
#include <iostream>

class Shape {
protected:
    float width, height;
public:
```

```
    void set_data(float a, float b) {
        width = a;
        height = b;
    }
};

class Rectangle : public Shape {
public:
    float area() {
        return (width * height);
    }
};

class Triangle : public Shape {
public:
    float area() {
        return (width * height / 2);
    }
};

int main() {

    Rectangle rect;
    Triangle tri;
    rect.set_data(5, 3);
    tri.set_data(2, 5);
    std::cout << rect.area() << std::endl;
    std::cout << tri.area() << std::endl;
    return 0;
}
```

9 Text File Handling

9.1 Write a program to write number 1 to 100 in a data file `notes.txt`. **Tip:** You can use library function `<fstream>`.

```
#include <iostream>
#include <fstream>

int main() {
    std::ofstream myfile;
    myfile.open("notes.txt");
    for (int i = 1; i <= 100; i++) { myfile << i << std::endl; }
    myfile << "Terminated.\n";
    myfile.close();
    return 0;
}
```

9.2 Write a program, which initializes a string variable to the content "It was the best of times, it was the worst of times." and outputs the string to the file `string_file.txt`. **Tip:** You can use library function

`<fstream>`.

```
#include <iostream>
#include <fstream>

int main() {
    // Create and open a text file
    std::ofstream MyFile("string_file.txt");

    // Write to the file
    MyFile << "It was the best of times, it was the worst of times.";

    // Close the file
    MyFile.close();
}
```

9.3 Write a program to read the content from a text file `text.txt`, count and display the number of characters present in it. **Tip:** You can use library function `<fstream>`.

```
#include <iostream>
#include <fstream>

int main() {
    std::ifstream fin("text.txt");
    char character; int count = 0;
    while (fin >> character){ ++count; }
    std::cout << count << " characters\n";
    return 0;
}
```

9.4 Write a program to count the number of blank present in a text file named `blank_spaces.txt`. **Tip:** You can use library functions `<fstream>` and `<string>`.

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::string line; int spaceCounter;
    std::ifstream myfile("blank_spaces.txt");
    if (myfile.is_open()) {
        std::string content((std::istreambuf_iterator<char>(myfile)),
                            (std::istreambuf_iterator<char>()));

        std::cout << content << std::endl;
        spaceCounter = 0;

        for (int i = 0; content[i] != '\0'; i++) {
            if (content[i] == ' ') { spaceCounter++; }
        }
```

```
        myfile.close();
    } else { std::cout << "Unable to open file"; }
    std::cout << "White spaces in the textfile: " << spaceCounter << std::endl;
    return 0;
}
```

9.5 Write a function to count number of words in a text file named `words.txt`. **Tip:** You can use library functions `<fstream>` and `<string>`.

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::string line;
    int spaceCounter;
    std::ifstream myfile("blank_spaces.txt");
    if (myfile.is_open()) {
        std::string content((std::istreambuf_iterator<char>(myfile)),
                            (std::istreambuf_iterator<char>()));

        std::cout << content << std::endl;

        spaceCounter = 0;

        for (int i = 0; content[i] != '\0'; i++) {
            if (content[i] == ' ') {
                spaceCounter++;
            }
        }
        myfile.close();
    } else std::cout << "Unable to open file";
    std::cout << "White spaces in the textfile: " << spaceCounter + 1 << std::endl;
    return 0;
}
```

9.6 Write a function to print the 10 most frequent words in the text file `story.txt`. **Tip:** Use the following libraries: `<fstream>`, `<string>`, `<map>`, `<vector>`, `<algorithm>` and `<cctype>`.

```
#include <iostream>
#include <map>
#include <vector>
#include <string>
#include <fstream>
#include <algorithm>
#include <cctype>

void Lower(std::string &word) {
    for_each(word.begin(), word.end(), [](char &c) { c = tolower(c); });
}
```

```cpp

bool LessThan(std::pair<std::string, size_t> left, std::pair<std::string, size_t> right){
    if (left.second == right.second) return (left.first < right.first);
    return (left.second < right.second);
}

int main() {
    std::map<std::string, size_t> words;
    std::ifstream inf("story.txt");
    std::string word;
    while (inf >> word) { Lower(word); ++words[word]; }

    // display original data
    std::vector<std::pair<std::string, size_t>> freq;
    for (auto m: words) freq.push_back(m);
    sort(freq.begin(), freq.end(), LessThan);
    // display new sorted data
    std::cout << "The 10 most frequently used words are:" << std::endl;
    for (int i = 0; i < 10; ++i) {
        std::cout << (freq.crbegin() + i)->first << " " << (freq.crbegin() + i)->second << " times" << std::endl;
    }
}
```

10 Pointers

10.1 Write a program that asks the user to enter integers as inputs to be stored in the variables `k` and `l` respectively. There are also two integer pointers named `ptrK` and `ptrL`. Assign the values of `k` and `l` to `ptrK` and `ptrL` respectively and display them.

```cpp
#include <iostream>

int main() {

    int k; int l;
    std::cout << "Enter value of K: "; std::cin >> k;
    std::cout << "Enter value of L: "; std::cin >> l;
    int *ptrK = &k; int *ptrL = &l;
    std::cout << "Value of ptrK is " << *ptrK << " stored in address " << ptrK << "\n";
    std::cout << "Value of ptrL is " << *ptrL << " stored in address " << ptrL << "\n";
    return 0;
}
```

10.2 Write a program to find the maximum value of an integral data set. The program will ask the user to input the number of data values in the set and each value. The program prints on screen a pointer that points to the maximum value.

```cpp
#include<iostream>

int main() {

    int n; int i; int max = 0;
    std::cout << "Enter number of values:"; std::cin >> n;
    std::cout << "Enter values in array:\n";
    int arr[n];

    for (i = 0; i < n; i++) { std::cin >> arr[i]; }
    for (int u = 0; u <= n; u++) { if (arr[u] > max) { max = arr[u]; } }

    int *pointer = &max;
    std::cout << "Largest integer value in the array is " << *pointer;
    return 0;
}
```